

HelixTrack

HelixTrack

JIRA의 자유 세계 대체 솔루션

요약

HelixTrack는 JIRA(및 Documents 확장 기능을 통해 Confluence)의 현대적이고 종합적인 오픈소스 대체 솔루션으로, 웹, 데스크톱, 모바일을 아우르는 멀티플랫폼 프로젝트 관리 및 이슈 추적 시스템입니다. Go 기반 마이크로서비스 백엔드 위에서 구동됩니다.

간략 설명

오픈소스 JIRA/Confluence 대체 솔루션. Go 마이크로서비스 백엔드 ("HelixTrack Core")는 프로젝트 및 이슈 추적을 위한 통합 REST API 인터페이스와 Confluence 스타일의 문서 작업 공간을 제공하며, HTTP/3 QUIC를 통해 웹, 데스크톱, 안드로이드, iOS 네이티브 클라이언트에 서비스를 제공합니다.

상세 설명

HelixTrack는 프로젝트 관리 및 이슈 추적 플랫폼으로, 엔지니어링 조직 대부분이 종속되어 있는 JIRA와 Confluence의 완전한 대체 솔루션을 목표로 하는 오픈소스 프로젝트입니다. 사용자가 소유하고 언제 어디서든 실행할 수 있는 소프트웨어로 재구성되었습니다. 이 시스템의 핵심은 **HelixTrack Core**로, Go와 Gin 프레임워크로 구현된 REST API 마이크로서비스입니다. 이슈 추적, 애자일/스크럼 보드, 팀 관리 기능과 함께 계층형 권한 엔진을 제공하며, 로컬 프로세스 내 엔진과 HTTP 기반 서비스를 교체 가능한 방식으로 구현해 동일한 인증 모델이 노트북 한 대부터 분산 클러스터까지 애플리케이션 코드 변경 없이 확장됩니다.

Core는 수십 개의 경로에 REST 인터페이스를 분산시키지 않고, 모든 기능을 단일 액션 라우팅 /do 엔드포인트로 집중시킵니다. 요청/응답 구조는 일관된 포맷(action/jwt/object/data 입력, errorCode/errorMessage/data 출

력)을 따르며, 모든 클라이언트는 동일한 간결한 규약을 사용합니다. 새로운 기능을 추가할 때도 새로운 URL을 정의하고 문서화하며 보안과 버전을 관리할 필요 없이 단순히 액션을 추가하기만 하면 됩니다.

Core는 HTTP/3 QUIC를 통해 통신하는 분리된 인증, 권한, 지역화 서비스를 통합하며, 이들은 별도의 머신이나 클러스터에서 실행될 수도 있고 테스트 환경에서는 완전히 비활성화할 수도 있습니다. 데이터는 개발 환경에서는 SQLite에 저장되어 손쉬운 설정을 지원하고, 운영 환경에서는 PostgreSQL에 저장됩니다. 또한 SQLCipher(AES-256)로 저장 시 암호화되어 민감한 프로젝트 데이터가 디스크에 기본적으로 안전하게 보호됩니다.

Documents V2 확장 기능을 통해 이슈 트래커는 완전한 지식 플랫폼으로 변모합니다. Confluence 스타일의 작업 공간으로, 스페이스, 페이지, 버전 관리, 템플릿, 실시간 WebSocket 협업, 분석 기능을 제공하며 위키와 이슈 트래커가 마침내 하나의 백엔드에서 통합 운영됩니다. 두 제품을 이어 붙인 방식이 아닌 진정한 통합 솔루션입니다.

Core를 중심으로 다양한 클라이언트 애플리케이션이 제공됩니다. Angular 웹 클라이언트, Tauri + Angular 데스크톱 클라이언트, 네이티브 안드로이드(Kotlin) 및 iOS(Swift) 앱, HarmonyOS와 Aurora OS 클라이언트, 그리고 스크린셰이퍼까지 지원됩니다. 모든 클라이언트는 동일한 백엔드와 통신하며, UDP 브로드캐스트를 통해 로컬 네트워크에서 자동으로 서버를 발견하므로 수동 설정 없이도 즉시 사용할 수 있습니다. 클라이언트 애플리케이션은 별도의 비공개 저장소에서 관리되며, 여기서는 제품 수준에서만 소개됩니다.

콘텐츠

왜 만들었는가

팀에게 진정한 오픈 소스이자 자체 호스팅 가능한 JIRA + Confluence 대체 솔루션을 제공하기 위해 — “자유로운 세계를 위해” — 벤더 종속 없이, 기업급 추적, 문서, 협업 기능을 하나의 오픈 소스 라이선스로 통합했습니다.

왜 혁신적인가

이 플랫폼은 두 개의 무거운 상용 제품 — 이슈 추적과 위키/문서 스택 — 을 하나로 통합한 오픈, 고성능, 자체 호스팅 가능한 솔루션으로 재탄생시켰으며, 기존 업체들이 제공하지 않았던 진정한 멀티 플랫폼 네이티브 클라이언트(웹, 데스크톱, 안드로이드, iOS, HarmonyOS, Aurora OS)를 단일 백엔드 계약으로 구동합니다. 진정한 혁신은 타협 없는 소유권입니다. HTTP/3 기반의 완전 분리형 마이크로서비스 설계와 저장 시 SQLCipher AES-256 암호화는 일반적으로 독점 SaaS에서만 제공되는 성능과 보안 수준을 여러 분이 직접 호스팅하는 시스템에서 구현합니다. 사용자 라이선스도, 벤더 종속도, 데이터 유출도 없습니다. 팀은 이미 익숙한 JIRA + Confluence 경험을 자체 하드웨어에서, 하나의 오픈 소스 라이선스로 누릴 수 있습니다.

무엇이 혁신적인가

- 통합 액션 기반 /do API — 하나의 엔드포인트, 하나의 엔벨로프, 액션 라우팅. 새로운 기능은 새로운 URL이 아닌 새로운 액션으로 추가되어 공격 표면, 클라이언트 코드, 문서화 부담을 단일 계약으로 통합합니다.
- HTTP/3 QUIC를 기본 서비스 간 전송 수단으로 — 현대적인 저지연, 연결 복원력 있는 네트워킹을 처음부터 서비스 간 통신에 적용, 사후 보완이 아닙니다.
- 로컬 인프로세스 구현과 HTTP 기반 서비스를 교체 가능한 권한 엔진, 선택적이며 독립 배포 가능한 Auth, Permissions, Localization 서비스 — 단일 프로세스든 클러스터든 동일한 인증 모델을 적용합니다.
- --space-root 플래그를 통한 멀티 스페이스 데이터 격리: 각 프로젝트는 고유한 데이터베이스와 자산 저장소를 가지며, 테넌트와 프로젝트는 쿼리 필터가 아닌 저장소 경계에서 분리됩니다.
- 저장 시 SQLCipher AES-256 암호화 — 민감한 프로젝트 데이터는 기본적으로 디스크에서 투명하게 보호됩니다.
- UDP 브로드캐스트를 통한 로컬 네트워크 자동 클라이언트-서버 발견 — 클라이언트는 수동 설정 없이 Core를 자동으로 찾습니다.
- Documents V2, 진정한 “Confluence 대체재” — 낙관적 잠금 병렬 편집, 충돌 감지, 전체 변경 이력을 갖춘 실시간 협업 문서로, 추적기와 동일한 백엔드에서 동작합니다.

가장 큰 기술적 도전 과제와 해결 방법

- 여섯 개의 클라이언트 플랫폼, 하나의 백엔드, 계약 드리프트 제로. 웹/Angular, 데스크톱/Tauri, 안드로이드/Kotlin, iOS/Swift, HarmonyOS, Aurora 클라이언트를 유지하는 것은 일반적으로 여섯 개의 서로 다른 API 통합을 의미하며, 이는 동기화되지 않은 상태로 퇴화하기 쉽습니다. 우리는 단일 액션 라우팅 /do API와 고정된 엔벨로프를 유일한 계약으로 설정하여 모든 클라이언트가 동일하게 이를 타겟팅하도록 했고, UDP 브로드캐스트 서비스 발견을 추가하여 클라이언트가 네트워크에서 Core를 수동 설정 없이 찾을 수 있도록 했습니다.
- 서비스 분리 시 지연 시간 부담 없애기. Auth, Permissions, Localization을 독립 배포 가능한 서비스로 분리하면 일반적으로 호출마다 네트워크 홉이 추가됩니다. 우리는 모든 서비스 간 호출에 HTTP/3 QUIC를 채택하여 이러한 홉을 빠르고 연결 복원력 있게 유지했으며, 각 서비스를 독립적으로 실행 가능하게 — 심지어 테스트 구성에서 완전히 비활성화할 수도 있게 — 만들어 서비스 분리는 배포 선택 사항이지 고정 비용이 아닙니다.
- **Confluence**급 협업에서 쓰기 손실 문제 방지. 실시간 다중 작성 편집은 충돌 쓰기를 유발할 수 있습니다. Documents V2는 낙관적 잠금 기반의 스페이스/페이지/버전 관리, 명시적 충돌 감지, 전체 변경 이력 백업, 실시간 WebSocket 동기화를 통해 일관성을 유지하는 협업을 구현했습니다. 이는 조용히 편집 내용을 덮어쓰는 대신 일관성을 유지합니다.
- 저장 시 암호화로 처리량 저하 방지. SQLCipher AES-256은 디스크 데이터를 보호하지만 쿼리마다 오버헤드를 발생시킵니다. 우리는 다층 캐싱(메모리 내 LRU와 Localization 서비스의 Redis)을 적용하여 다

국어 조회와 같은 핫 패스는 빠르게 유지하면서 데이터는 암호화된 상태로 보관합니다.

콘텐츠

기술 스택

- **Go + Gin** — 고처리량, 저지연 HTTP 서비스를 단일 바이너리로 배포할 수 있어 선택되었습니다. Core의 REST API, JWT/CORS 미들웨어, 그리고 전체 시스템의 프론트 역할을 하는 액션 라우팅 /do 라우터를 포함합니다.
- **HTTP/3 QUIC** — Core와 인증/권한/로컬라이제이션 서비스 간의 통신에 사용됩니다. QUIC의 멀티플렉싱 및 연결 마이그레이션 설계는 TCP에서 발생하는 지연 문제를 해결하고 불안정한 링크에서도 안정성을 보장합니다.
- **PostgreSQL(프로덕션) / SQLite(개발)** — 대규모 추적 및 문서 스키마를 지원하는 단일 관계형 모델로, 두 엔진 모두에서 사용됩니다. SQLite는 로컬 개발 환경을 제로 셋업 및 파일 기반으로 유지하며, 프로덕션에서는 전용 production 컴포즈 프로파일을 통해 Postgres로 전환됩니다.
- **SQLCipher(AES-256)** — 데이터베이스 수준에서 투명한 암호화를 제공하여 민감한 프로젝트 데이터를 보호합니다. 애플리케이션 계층의 암호화 없이도 쿼리 작성 방식 변경 없이 보안이 유지됩니다.
- **Redis** — 로컬라이제이션 서비스 내 인메모리 LRU 캐시 뒤에 공유 캐시 계층으로 사용되어, 암호화 오버헤드에도 불구하고 핫 멀티언어 조회를 빠르게 유지하는 이중 캐시 구조를 제공합니다.
- **Uber Zap + Lumberjack** — 구조화된 로깅과 메모리 효율적인 로그 회전을 지원하여, 프로덕션 환경에서도 무한정 로그 증가를 방지하면서 시스템 가시성을 유지합니다.
- **golang-jwt / JWT** — 무상태 인증 메커니즘으로 선택되었습니다. 서명된 토큰은 모든 /do 요청의 jwt 필드에 포함되어 모든 클라이언트에서 일관된 인증을 제공합니다.
- **Angular 19(+Material, RxJS)** — 반응형 컴포넌트 기반 브라우저 클라이언트를 구현하기 위해 선택되었으며, 성숙한 Material 디자인 시스템을 즉시 활용할 수 있습니다.
- **Tauri 2.0 + Rust + Angular** — Angular UI를 Rust 기반 웹뷰에 재사용하여 전체 브라우저 런타임을 번들링하지 않고도 경량 네이티브 데스크톱 셸을 제공합니다.
- **Kotlin(안드로이드) / Swift + SwiftUI(iOS)** — 모바일 사용자에게 래핑된 웹뷰가 아닌 진정한 네이티브 플랫폼별 클라이언트를 제공합니다.
- **Docker / Docker Compose(Podman 호환)** — 재현 가능한 컨테이너화 배포를 위해 선택되었으며, /health 체크가 통합되어 있으며 Podman 호환성을 유지해 데몬이나 특정 벤더 의존성을 배제합니다.
- **Testify(Go); Cypress/Playwright/Karma+Jasmine(클라이언트)** — 백엔드 계약 및 클라이언트 UI를 독립적으로 검증하는 계층형 자동화 테스트로, 단일 백엔드/다중 클라이언트 아키텍처에 맞춰 설계되었습니다.

현황 및 투명성 안내

- **현황:** 베타. HelixTrack Core는 작동 중인 REST API 마이크로서비스입니다. **Documents V2** 확장 기능은 약 95% 완료되었으며, 알려진 데이터베이스 필드 매핑 문제가 있어 아직 공식 출시된 상태로 간주되지 않습니다.
- **라이선스:** 미정. CLAUDE.md에는 MIT로 명시되어 있으나, 공식 기록인 core/LICENSE 파일은 Apache 2.0입니다. 이 불일치는 라이선스 확정 전에 반드시 해결되어야 합니다.
- 프로젝트 README에 언급된 성능 수치(예: 초당 50,000건 이상의 요청 처리, 밀리초 미만 쿼리 응답 시간)는 독립적으로 검증된 벤치마크가 아닌 설계/마케팅 목표이므로, 위 내용에서는 생략되었습니다.
- 클라이언트 애플리케이션(웹, 데스크톱, 안드로이드, iOS, Aurora, HarmonyOS)은 비공개 저장소에 있으며, 제품 수준에서만 설명됩니다.

우선 순위 등급: Helix-주력 및 Helix-Track 제품 라인의 플래그십 모델 — 모든 Server Factory 프로젝트보다 우선합니다.